

CRITÉRIOS DE AVALIAÇÃO DO TRABALHO PRÁTICO DA DISCIPLINA DE APRENDIZAGEM AUTOMÁTICA

Licenciatura em Engenharia de Sistemas e Tecnologias informáticas
Departamento de Engenharia Eletrotécnica

Instituto Superior de Engenharia - Universidade do Algarve

Pedro Cardoso
pcardoso@ualg.pt

Última atualização: 6 de outubro de 2025

Conteúdo

1	Introdução	1
2	Quem faz o trabalho prático?	2
3	Que trabalho prático?	2
3.1	Parte 1: Kaggle - Competição de aprendizagem automática	2
3.2	Parte 2: Aplicação para Detecção de Língua Gestual	3
4	Quais os prazos de entrega?	6
4.1	Parte 1: Kaggle - Competição de aprendizagem automática	6
4.2	Parte 2: Aplicação para Detecção de Língua Gestual	6
5	Quais as competências do Aluno Aprovado?	7
5.1	Parte 1: Kaggle - Competição de aprendizagem automática	7
5.2	Parte 2: Aplicação de aprendizagem automática	7
6	Classificação & Critérios de Correção	7
6.1	Parte 1: Kaggle - Competição de aprendizagem automática	7
6.2	Parte 2: Aplicação para Detecção de Língua Gestual	8
6.3	Avaliação	8

1 Introdução

Da unidade curricular de Aprendizagem Automática espera-se que os alunos adquiram competências na área de aprendizagem automática, nomeadamente na análise de dados, desenvolvimento de modelos de aprendizagem automática, afinação de parâmetros, avaliação de modelos, etc.

Neste contexto, o trabalho prático (TP) tem como objetivo principal permitir aos alunos aplicar os conhecimentos adquiridos na unidade curricular. A avaliação por TP é composta por 2 partes, cada uma com objetivos específicos.

É muito importante que os alunos leiam este documento com atenção e que esclareçam todas as dúvidas com o docente da UC.

É ainda muito importante que os alunos leiam o modo como será feita a defesa dos TP, nomeadamente a forma como será descontado pontos em caso de não saberem responder a questões sobre o trabalho que desenvolveram (ver Secção 6).

2 Quem faz o trabalho prático?

- Todos os alunos têm de fazer o trabalho prático.
- A nota mínima na componente prática é de 7 (sete) valores.
- Cada trabalho prático deve ser realizado por grupos de até 2 alunos - acordados com o docente.

3 Que trabalho prático?

O TP é composto por 2 partes descritas a seguir.

3.1 Parte 1: Kaggle - Competição de aprendizagem automática

Considere a competição privada "Preço de Carros Usados", disponível em <https://www.kaggle.com/t/d726ef894a1a42208a14b1f6d611bf61>.

Para realizar o trabalho, siga as instruções:

1. Leia atentamente o enunciado no Kaggle, incluindo a descrição dos dados, as regras e a métrica de avaliação. Faça o download de todos os ficheiros disponibilizados.
2. Construa modelos para prever o *target*. Como restrição, utilize apenas algoritmos da lista seguinte:
 - Regressão Linear
 - K-Nearest Neighbors (KNN)
 - Árvores de Decisão
 - Random Forest e variantes (e.g., XGBoost)
 - Support Vector Machines (SVM)
 - Redes Neurais
3. Avalie e ajuste os modelos. Utilize *cross validation* para uma avaliação robusta e *grid search* para otimização de hiperparâmetros. Sempre que aplicável, fixe as sementes aleatórias para garantir reprodutibilidade.
4. Submeta os resultados no Kaggle. Tenha em atenção que existem, no máximo, 5 submissões por dia.
5. Entregue (na tutoria da UC) o ficheiro TP1_<xxxxxx>_<xxxxxx>.ipynb, em que:
 - o ficheiro deve estar em formato Jupyter Notebook (.ipynb). Se necessário, comprima-o num .zip antes de submeter;
 - <xxxxxx> corresponde aos números de aluno;
 - o notebook deve documentar todo o processo de treino (e.g., *grid search*, *cross validation*, hiperparâmetros usados), incluindo outputs visíveis, gráficos e comentários explicativos;
 - o código deve executar do início ao fim, de forma sequencial e sem erros;
 - deve incluir uma captura de ecrã com a melhor pontuação obtida no quadro de líderes da competição;
 - a entrega de um notebook sem explicação do processo de treino e dos hiperparâmetros usados será considerada como não entrega ou será fortemente penalizada.

3.2 Parte 2: Aplicação para Detecção de Língua Gestual

Nesta parte do trabalho prático pretende-se que desenvolva uma aplicação que permita detetar em tempo real os caracteres que um utilizador faz com língua gestual. Para tal, deve usar o dataset *SignAlphaSet* [Garg et al., 2025] que pode ser encontrado em <https://data.mendeley.com/datasets/8fmvr9m98w/1>.

Este conjunto de dados é composto por cerca de 26.000 imagens que representam gestos manuais da Linguagem Gestual Americana (ASL) correspondentes às letras do alfabeto inglês (A-Z). Na realidade as letras J e Z estão presentes mas correspondem a gestos com movimentos pelo que poderão não ser classificados correctamente.

O conjunto está organizado em 26 pastas, cada uma rotulada com a respetiva letra, contendo múltiplas amostras de imagens para garantir variabilidade na posição das mãos, iluminação e diferenças individuais entre mãos. A Figura 1 mostra algumas imagens do dataset.

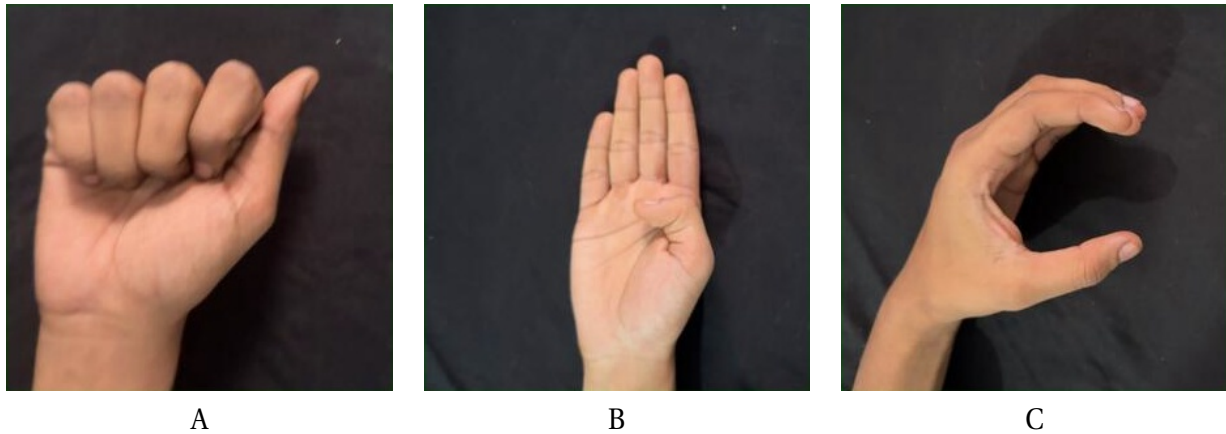


Figura 1: Exemplo de imagens do dataset *SignAlphaSet*

O desenvolvimento deve estar dividido em 4 fases:

Fase 1: Préprocessamento e Extração de Landmarks Esta fase consiste na transformação do dataset *SignAlphaSet* de imagens RGB para um dataset estruturado de *landmarks* das mãos. O objetivo é reduzir a dimensionalidade dos dados e extrair características relevantes para a classificação.

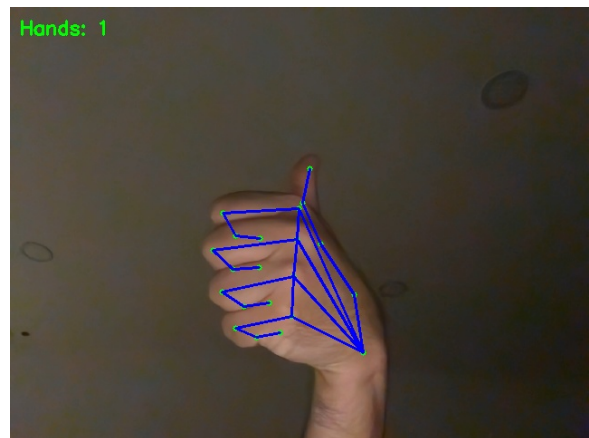
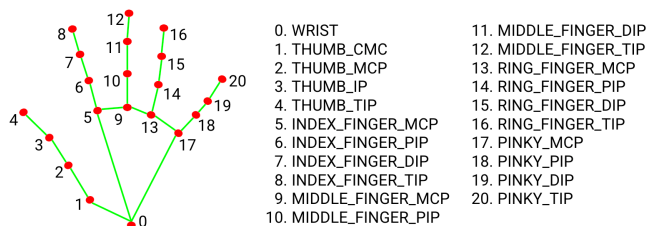


Figura 2: Exemplo de *landmarks* das mãos

Objetivo: Converter cada imagem do dataset original numa representação compacta baseada em pontos-chave (*landmarks*) das mãos, mantendo a informação essencial para a classificação dos gestos.

Implementação: Para extrair os *landmarks* das mãos, utilize os ficheiros fornecidos que utilizam a biblioteca mediapipe [Google Research, 2025]:

- `hand_landmark_extractor.py`: Contém a classe `HandLandmarkExtractor` que processa imagens e extrai as coordenadas dos *landmarks* das mãos.
- `test_camera.py`: Exemplo de utilização da classe para processamento em tempo real via câmara.

Processo de Extração:

1. Itere sobre todas as pastas do dataset *SignAlphaSet* (A-Z).
2. Para cada imagem, utilize a classe `HandLandmarkExtractor` para extrair os *landmarks* das mãos.
3. Cada *landmark* devolvido pela classe contém a informação da mão (esquerda ou direita) e as coordenadas dos pontos-chave da mão, representado por coordenadas tridimensionais (x, y, z) .
4. A letra correspondente (*label*) é a letra que o gesto representa e obtém-se através da pasta onde a imagem se encontra.

Resultado Esperado: Um novo dataset estruturado contendo:

- Coordenadas dos *landmarks* das mãos (21 pontos por mão, num total de 63 valores)
- Identificação da mão (esquerda/direita)
- *Label* da letra correspondente (A-Z)

Fase 2: Desenvolvimento e Treino do Modelo de Classificação Esta fase consiste no desenvolvimento de um modelo de aprendizagem automática para classificar os gestos de língua gestual baseado nos *landmarks* extraídos na fase anterior.

Objetivo: Criar um modelo de classificação capaz de identificar corretamente as letras do alfabeto (A-Z) a partir das coordenadas dos *landmarks* das mãos.

Metodologia:

1. **Preparação dos Dados:** Utilize o dataset de *landmarks* criado na Fase 1, dividindo-o em conjuntos de treino, validação e teste.
2. **Seleção de Algoritmos:** Teste diferentes algoritmos de classificação, limitando-se aos métodos abordados nas aulas:
 - *Random Forest* e variantes (e.g., XGBoost)
 - *Support Vector Machines (SVM)*
 - *K-Nearest Neighbors (KNN)*
 - *Redes Neurais*
 - *Árvores de Decisão*
3. **Otimização de Hiperparâmetros:** Para cada algoritmo, utilize técnicas de *grid search* e *cross validation* para encontrar os melhores parâmetros.
4. **Avaliação:** Compare o desempenho dos diferentes modelos usando métricas apropriadas (accuracy, precision, recall, F1-score). Em caso de empate, escolha o modelo que tem o tempo de inferência mais baixo.

5. Desenvolva uma aplicação que carregue o modelo treinado e o teste utilizando os ficheiros referidos para extrair os *landmarks*; em seguida, envie-os para o modelo para obter a respetiva previsão. Caso nesta fase o desempenho fique aquém do esperado, considere aumentar o *dataset* com amostras adicionais recolhidas pelos elementos do grupo.

Resultado Esperado: Um modelo treinado e otimizado, guardado em disco (formato .pkl), pronto para ser utilizado na aplicação web.

Fase 3: Desenvolvimento da API Flask Esta fase consiste na criação de uma API web que disponibilize o modelo de classificação desenvolvido na Fase 2 através de uma interface REST.

Objetivo: Criar uma aplicação Flask que exponha o modelo de classificação através de uma API HTTP, permitindo que aplicações cliente possam enviar dados de *landmarks* e receber a classificação do gesto correspondente.

Implementação:

1. **Carregamento do Modelo:** A aplicação Flask deve carregar o modelo treinado (ficheiro .pkl) criado na Fase 2.
2. **Endpoint da API:** Criar um endpoint que aceite dados de *landmarks* das mãos e devolva a classificação.
3. **Processamento:** O endpoint deve:
 - Receber as coordenadas dos *landmarks* (formato JSON)
 - Aplicar o mesmo pré-processamento usado no treino (se aplicável)
 - Executar a classificação usando o modelo carregado
 - Devolver a letra identificada e, se aplicável, a confiança da predição

Especificações Técnicas:

- **Framework:** Flask (Python)
- **Formato de Entrada:** JSON com coordenadas dos *landmarks*
- **Formato de Saída:** JSON com a letra identificada e score de confiança
- **Tratamento de Erros:** Implementar validação de dados e tratamento de exceções

Resultado Esperado: Uma API funcional e documentada, pronta para ser consumida por aplicações cliente¹.

Fase 4: Desenvolvimento da Aplicação Cliente Esta fase consiste na criação de uma aplicação cliente que permita aos utilizadores interagir com o sistema de classificação de língua gestual através de uma interface amigável.

Objetivo: Desenvolver uma aplicação que capture imagens/vídeo da mão do utilizador, extraia os *landmarks* correspondentes e comunique com a API Flask para obter a classificação do gesto em tempo real.

Opções de Implementação:

1. **Aplicação Nativa Python:**

- Utilize a câmara do computador para captura em tempo real
- Integre a classe `HandLandmarkExtractor` para extrair *landmarks*

¹Numa das aulas é apresentado um exemplo de uma aplicação Flask que disponibiliza uma API para classificar iris com base num modelo de aprendizagem automática.

- Implemente interface gráfica (e.g., Kivy, Tkinter, PyQt) para visualização (opcional)
- Estabeleça comunicação HTTP com a API Flask

2. Aplicação Móvel:

- Desenvolva aplicação para smartphone/tablet
- Implemente captura de imagem/vídeo através da câmara do dispositivo
- Integre processamento de *landmarks* ou envie dados para API
- Crie interface touch-friendly para melhor experiência do utilizador

Funcionalidades Obrigatórias:

- **Captura de Dados:** Captura de imagens/vídeo da mão do utilizador
- **Extração de Landmarks:** Processamento das imagens para obter coordenadas dos *landmarks*
- **Comunicação com API:** Envio dos dados para a API Flask e receção da classificação
- **Visualização:** Apresentação clara do resultado da classificação
- **Feedback Visual:** Indicação da confiança da predição e possíveis erros

Resultado Esperado: Uma aplicação cliente funcional que demonstre o sistema completo de classificação de língua gestual em tempo real.

A entrega deve incluir:

1. um notebook onde é apresentado o código desenvolvido no processo de afinação do modelo de aprendizagem automática (.ipynb)
2. um ficheiro com o modelo de aprendizagem automática escolhido de entre os testados para ser usado na aplicação (.pkl)
3. um ficheiro com a aplicação Flask desenvolvida (.py)
4. um ficheiro com a aplicação cliente desenvolvida.

4 Quais os prazos de entrega?

A entrega deve ser feita na tutoria da UC. Os prazos de entrega são os especificados a seguir.

4.1 Parte 1: Kaggle - Competição de aprendizagem automática

O trabalho dever ser submetido no moodle até ao dia anterior ao exame de época normal. Caso não entreguem até à data indicada, podem entregar até ao dia anterior ao do Exame de Recurso, sendo que neste caso penalizados em 1 valor por semana de atraso (a contar do 1º dia da semana de atraso). Apesar da data indicada, sugere-se a entrega até 5 de dezembro de 2025.

4.2 Parte 2: Aplicação para Detecção de Língua Gestual

O trabalho dever ser submetido no moodle até ao dia anterior ao exame de época normal. Caso não entreguem até à data indicada, podem entregar até ao dia anterior ao do Exame de Recurso, sendo que neste caso penalizados em 1 valor por semana de atraso (a contar do 1º dia da semana de atraso).

Apesar da data indicada, sugere-se a entrega até 12 de dezembro de 2025.

5 Quais as competências do Aluno Aprovado?

5.1 Parte 1: Kaggle - Competição de aprendizagem automática

1. Prática no desenvolvimento de modelos de aprendizagem automática.
2. Prática na afinação de parâmetros de modelos de aprendizagem automática, nomeadamente usando *grid search* e *cross validation*.
3. Prática na análise de resultados de modelos de aprendizagem automática.
4. Prática na submissão de resultados para uma competição de aprendizagem automática.
5. Prática na análise de resultados de uma competição de aprendizagem automática.
6. Prática na escrita de um relatório técnico.
7. Prática na apresentação de resultados.

5.2 Parte 2: Aplicação de aprendizagem automática

- Prática de tratamento de imagens.
- Desenvolver um modelo de aprendizagem automática para classificar caracteres de língua gestual recorrendo a diferentes modelos e diferentes parâmetros, p.e., *Random Forest*, *SVM*, *KNN*, *Redes Neurais*, etc.
- Desenvolver uma aplicação Flask que carregue o modelo desenvolvido e que disponibilize uma API que receba uma imagem e devolva a classificação do carácter.
- Desenvolver uma aplicação cliente (nativa ou móvel) que permita ao utilizador capturar uma imagem e que devolva a classificação do carácter.

6 Classificação & Critérios de Correção

A classificação final do TP é calculada de acordo com a seguinte fórmula:

$$TP = 0.3 \times \text{Parte 1} + 0.7 \times \text{Parte 2} \quad (1)$$

6.1 Parte 1: Kaggle - Competição de aprendizagem automática

Os critérios de correção terão em conta os seguintes aspetos:

1. Desenvolvimento de modelos de aprendizagem automática, nomeadamente os modelos testados e os parâmetros usados. Nesta fase será valorizado o uso de diferentes modelos e o uso de técnicas de afinação de parâmetros como seja *grid search* e *cross validation*. – 65%
2. Análise dos resultados obtidos, nomeadamente a análise das métricas de avaliação dos modelos desenvolvidos. – 20%
3. Submissão dos resultados na plataforma do Kaggle. – 15%

6.2 Parte 2: Aplicação para Detecção de Língua Gestual

Os critérios de correção terão em conta os seguintes aspetos:

1. **Fase 1 (Preprocessamento e Extração de Landmarks):** Qualidade do pipeline de extração de *landmarks* e criação do dataset estruturado (organização, tratamento de erros, consistência dos dados). - 20%
2. **Fase 2 (Modelo de Classificação):** Desenvolvimento e treino do modelo para classificar caracteres de língua gestual; valorizam-se a experimentação com diferentes algoritmos, a afinação de hiperparâmetros via *grid search* e *cross validation*, a análise das métricas e a reprodutibilidade. - 50%
3. **Fase 3 (API Flask):** Implementação de uma API que carregue o modelo treinado e disponibilize um endpoint que receba *landmarks* e devolva a classificação (com validação de dados e tratamento de erros). - 15%
4. **Fase 4 (Aplicação Cliente):** Desenvolvimento de uma aplicação cliente (nativa ou móvel) que capture imagem/vídeo, extraia *landmarks* e consuma a API, apresentando o resultado em tempo real de forma clara. - 15%

6.3 Avaliação

A avaliação de cada um dos TP será feita de acordo com os seguintes critérios:

- Todos os alunos começam com 20 valores, descontado o que não tenha sido feito corretamente (e.g., não ter seguido as instruções, não ter implementado a funcionalidade, não ter feito *grid search*, não ter feito *cross validation*, não ter descrito o processo de treino, não ter descrito os hiperparâmetros usados, etc.).
- O aluno deve ser capaz de responder a todas as questões sobre o trabalho que desenvolveu. Nesse sentido:
 - À primeira questão que o aluno não souber responder sobre o trabalho que desenvolveu, perde 1 valor.
 - À segunda questão que o aluno não souber responder sobre o trabalho que desenvolveu, perde 3 (= 1 + 2) valores.
 - À terceira questão que o aluno não souber responder sobre o trabalho que desenvolveu, perde 7 (= 1 + 2 + 4) valores.
 - À quarta questão que o aluno não souber responder sobre o trabalho que desenvolveu, perde 15 (= 1 + 2 + 4 + 8) valores.
 - ...
- Plágio, geradores automáticos de código, etc.: Será atribuída classificação negativa trabalhos que:
 - sejam cópias uns dos outros (0 valores - sem possibilidade de melhoria)
 - não sejam realizados pelos alunos que os entregam (0 valores - sem possibilidade de melhoria)
 - não sejam apresentados na sessão de defesa (0 valores)
 - sejam gerados parcial ou totalmente por geradores automáticos de código (0 valores - sem possibilidade de melhoria)

NÃO EXISTE MELHORIA DE NOTA

**Em caso de dúvidas contactar o professor:
pcardoso@ualg.pt**

Referências

- [Garg et al., 2025] Garg, B., Kasar, M., Kashyap, A., Vats, A., Sharma, G., and Hange, A. (2025). Signalphaset.
- [Google Research, 2025] Google Research (2025). Mediapipe — cross-platform, customizable ml solutions for live and streaming media. <https://developers.google.com/mediapipe/>. Accessed: 2025-10-05.